

基于协同过滤的电影推荐系统技术报告

周四 | 第 24 组

张伟俊 陈思思 蒋沛文 刘瑞凯

项目：A06 电影推荐系统（协同过滤）

数据集：MovieLens ml-latest-small

主要技术：Python、Pandas、NumPy、Matplotlib、Seaborn、scikit-learn、协同过滤

摘要

随着在线视频平台和影评社区的发展，用户面对的电影数量迅速增加，如何从大量候选电影中筛选出符合用户兴趣的内容，成为推荐系统中的典型问题。本项目以 MovieLens ml-latest-small 公开数据集为基础，围绕“电影推荐系统（协同过滤）”主题，完成了从数据清洗、探索性分析、稀疏矩阵处理到模型构建和实验评估的完整流程。项目首先对 ratings、movies、tags、links 四类原始数据进行格式统一、异常值处理、缺失值处理和去重，并构建用户-电影评分矩阵。随后通过用户评分分布、电影评分分布、评分时间趋势和矩阵密度热力图等可视化方式分析数据特征，并基于活跃用户和热门电影筛选策略降低评分矩阵稀疏性。

在模型方面，项目实现了两个基于物品的协同过滤模型：其一是使用 NumPy 手写余弦相似度的基线模型 `baseline_numpy_item_cf`；其二是 `src/cf_recommender.py` 中的主模型 `sklearn_centered_item_cf`。当前版本在主模型中进一步加入用户均值中心化、相似度 shrinkage、最低共评人数约束以及轻量 Top-N 重排序层，以降低低置信相似度带来的噪声，并改善推荐列表排序质量。实验采用按用户划分训练集和测试集的方式，使用 RMSE、MAE、Weighted_RMSE、Weighted_MAE、Precision@K、Recall@K 和 HitRate@K 对评分预测与 Top-N 推荐效果进行综合评价。

关键词：电影推荐；协同过滤；用户-物品矩阵；余弦相似度；Top-N 推荐

1. 引言

推荐系统是人工智能和数据挖掘领域的重要应用方向之一。对于电影、音乐、新闻、电商商品等内容平台而言，用户通常无法主动浏览全部候选内容，因此系统需要根据用户历史行为、物品属性或群体偏好生成个性化推荐结果。电影推荐是推荐系统中的经典场景：一方面，电影本身具有类型、标签、年代、演员等丰富内容特征；另一方面，用户对电影的评分行为能够形成典型的用户-物品交互矩阵，适合用于协同过滤算法研究。

协同过滤的基本思想是：如果两个用户过去喜欢相似的电影，则他们未来可能继续喜欢相似的电影；或者如果两部电影被相似的一批用户给予相近评分，则它们在偏好空间中也可能相似。本项目采用后者，即基于物品的协同过滤。相比基于用户的协同过滤，基于物品的方法在电影推荐场景中通常更稳定，因为电影集合相对固定，而用户行为可能持续变化。通过计算电影之间的相似度，可以根据某个用户已经评分的电影推断其对未评分电影的兴趣程度。

本项目的目标是构建一个可复现的电影推荐系统实验流程，重点不只是给出推荐结果，还包括数据来源说明、数据清洗、可视化分析、稀疏矩阵处理、模型实现、对比实验和结果讨论。根据课程任务书要求，项目使用 Python AI 相关库构建模型，并完成训练集/测试集划分与评价指标计算，同时包含基线模型与优化后主模型的全量对比实验。

2. 数据描述与预处理

本项目使用 MovieLens ml-latest-small 数据集。该数据集由 GroupLens Research 发布，常用于推荐系统教学与实验。原始数据主要包含四个 CSV 文件：`ratings.csv` 记录用户对电影的评分及时间戳，`movies.csv` 记录电影编号、标题和类型，`tags.csv` 记录用户给电影添加的自由文本标签，`links.csv` 提供 MovieLens 电影编号与外部电影数据库编号之间的映射关系。其中，评分数据是本项目建模的核心，电影标题数据用于展示推荐结果。

数据清洗过程由 `src/data_clean.py` 完成，主要包括以下步骤。首先，对 `userId`、`movieId`、`imdbId`、`tmdbId` 等编号字段进行数值化和整数化处理，避免隐藏空格、BOM 字符或异常格式影响后续关联。其次，对 `rating` 字段进行浮点化处理，过滤超出 0 到 5 范围的异常评分，并将不符合半星粒度的评分修正到最近的 0.5 档。再次，对 `timestamp` 字段统一转换为可读的 UTC 时间格式，方便进行时间趋势分析。对于缺失值，`tags` 文件中的缺失标签填充为空字符串，`movies` 文件中的缺失 `genres` 填充为 `Unknown`。最后，删除完全重复行，并针对同一用户对同一电影多次评分的情况保留最新一条记录。清洗后的文件保存到 `data/cleaned_data` 目录，同时生成清洗日志，保证处理过程可追溯。

在完成基础清洗后，项目通过 `src/data_processing.py` 将评分数据透视为用户-电影评分矩阵。矩阵的行表示用户，列表示电影，单元格表示用户对电影的评分，未评分位置记为缺失值。原始矩阵共有 610 名用户、9724 部电影和 100836 条评分，矩阵单元数为 5931640，稀疏率达到 98.30%。这说明绝大多数用户只评价了很小一部分电影，直接在完整矩阵上进行协同过滤会受到严重稀疏性的影响。

为缓解这一问题，项目在 `notebooks/analysis.ipynb` 中进行了稀疏矩阵分析和过滤阈值选择。通过比较多个候选阈值后，最终采用 `user>=30`、`movie>=15` 的筛选策略，即保留评分数不少于 30 的活跃用户和评分数不少于 15 的热门电影，并进行迭代过滤直到矩阵规模稳定。过滤后矩阵规模为 475 名用户、1617 部电影和 70454 条评分，稀疏率下降到 90.83%。虽然矩阵仍然稀疏，但相比原始矩阵，评分密度从 1.70% 提升到 9.17%，更适合后续协同过滤建模。

表 1 过滤前后评分矩阵统计

指标	过滤前	过滤后
用户数	610	475
电影数	9724	1617
评分数	100836	70454
矩阵单元数	5931640	768075
稀疏率	98.30%	90.83%
密度	1.70%	9.17%
用户平均评分数	165.30	148.32
电影平均评分数	10.37	43.57

原始矩阵评分分布

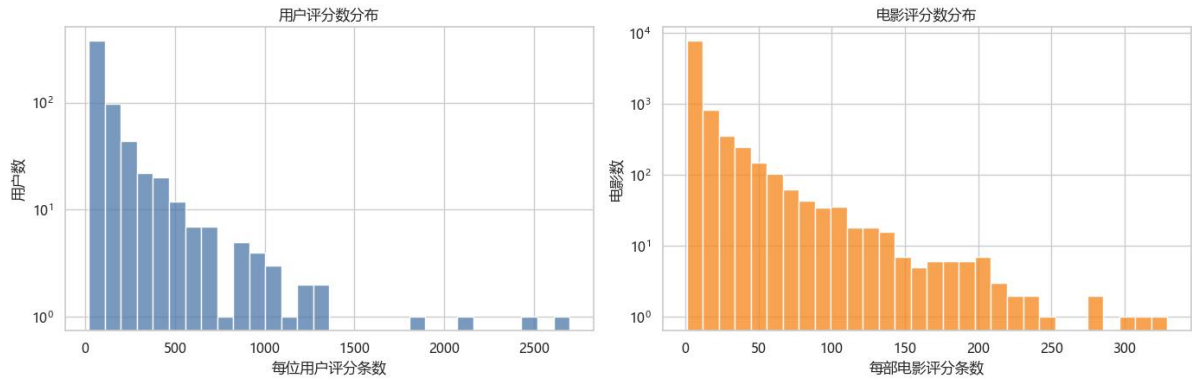


图 1 用户评分数与电影评分数分布

该图来自 analysis.ipynb，用于展示用户活跃度和电影流行度的长尾分布，是选择过滤阈值的重要依据。

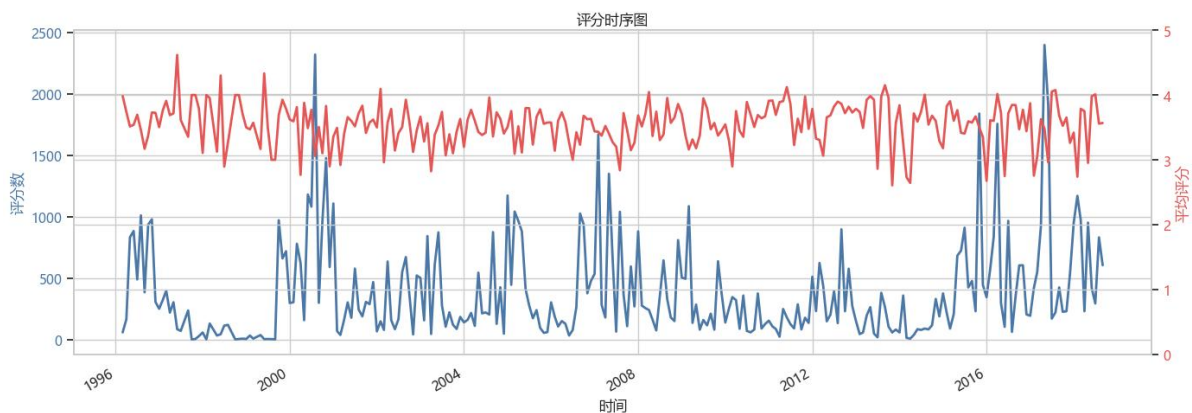


图 2 评分时间趋势图

该图展示不同月份评分数量和平均评分的变化，可用于说明数据在时间维度上的活跃度差异。

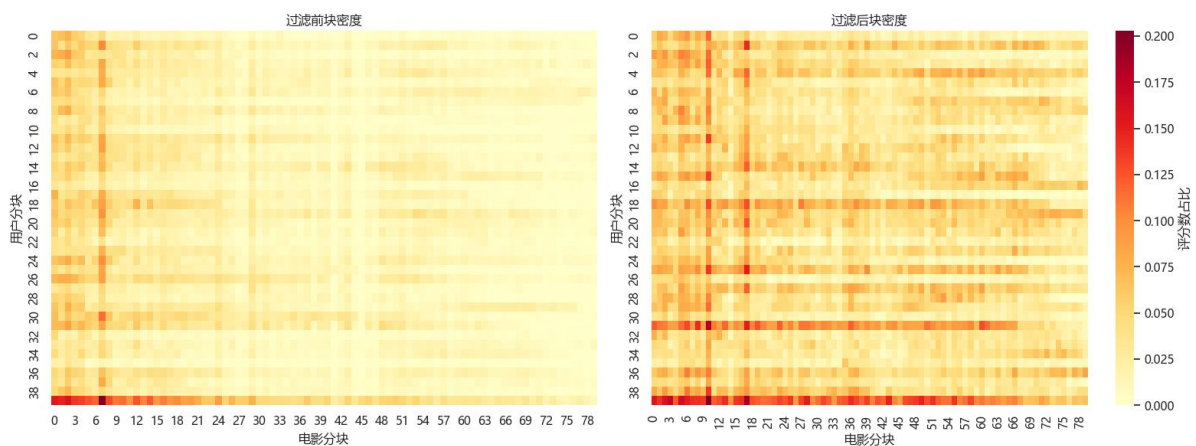


图 3 过滤前后矩阵块密度热力图

该图用于观察评分矩阵在过滤前后的密度变化，说明筛选活跃用户和热门电影后，高密度区域更加集中。

3. 方法模型

本项目采用基于物品的协同过滤算法。设用户-电影评分矩阵为 (R) ，其中 $(R_{\{u,i\}})$ 表示用户 (u) 对电影 (i) 的评分。对于任意两部电影 (i) 和 (j) ，可以将它们在所有用户上的评分视为两个向量，使用余弦相似度衡量两部电影在用户偏好空间中的接近程度：

$$sim(i, j) = \frac{\sum_u R_{u,i} R_{u,j}}{\sqrt{\sum_u R_{u,i}^2} \sqrt{\sum_u R_{u,j}^2}}$$

当需要预测用户 (u) 对未评分电影 (i) 的兴趣时，模型会从用户已经评分的电影集合中选取与电影 (i) 最相似的 Top-K 个邻居，并根据相似度加权计算预测评分。基线形式可表示为：

$$\hat{R}_{u,i} = \frac{\sum_{j \in N_k(i)} sim(i, j) R_{u,j}}{\sum_{j \in N_k(i)} |sim(i, j)|}$$

其中，(N_k(i)) 表示与目标电影 (i) 最相似的 K 部用户已评分电影。最终系统对用户所有未评分电影计算预测分数，并选取得分最高的 Top-N 电影作为推荐列表。

项目实现了两个模型进行对比。第一个模型是 src/model.py 中的 baseline_numpy_item_cf。该模型使用 NumPy 手写余弦相似度计算过程，先将缺失评分填充为 0，再通过矩阵乘法得到电影之间的相似度矩阵。该模型结构简单、透明，适合作为基线算法，能够帮助理解协同过滤的核心流程。

第二个模型是 src/cf_recommender.py 中的 sklearn_centered_item_cf，也是当前项目的主模型。该模型使用 sklearn.metrics.pairwise.cosine_similarity 计算物品相似度，并在计算相似度前按用户均值对评分矩阵进行中心化，以削弱不同用户评分尺度的差异。当前版本还对相似度加入共评人数置信度约束：共同评分用户越少，相似度越容易出现偶然偏高，因此模型使用 shrinkage 将低共评电影对的相似度向 0 收缩，并过滤低于最低共评人数阈值的电影对。

$$\hat{R}_{u,i} = \bar{R}_u + \frac{\sum_{j \in N_k(i)} sim(i, j) (R_{u,j} - \bar{R}_u)}{\sum_{j \in N_k(i)} |sim(i, j)|}$$

其中，({R}_u) 表示用户 (u) 的平均评分。该模型在评分预测阶段仍保持纯评分语义，只使用中心化评分和收缩后的物品相似度，并将预测结果裁剪到数据集中实际评分范围内。生成推荐列表时，模型再叠加轻量排序层，将电影评分人数和候选电影与用户历史电影的相似度证据强度作为小权重辅助信号，用于提升 Top-N 推荐排序的稳定性。

4. 实验设计与评价指标

实验逻辑由 src/evaluation.py 统一实现。为了公平比较两个模型，项目先对过滤后的评分矩阵进行同一次训练集/测试集划分。划分方式是按用户随机抽取一部分已评分电影作为测试集，并在训练矩阵中将这此评分隐藏为缺失值。这样，模型只能基于训练矩阵中的历史评分生成预测，再与测试集中被隐藏的真实评分进行比较。实验默认测试集比例为 0.2，随机种子为 42，保证结果可复现。notebooks/model_evaluation_visualization.ipynb 保留 20 用户快速演示用于说明流程，但正式结论以 475 个可评估用户、14093 条测试评分的全量评估为准。

评价指标分为两类。第一类是评分预测误差，包括 RMSE、MAE 以及按测试评分数加权汇总的 Weighted_RMSE 和 Weighted_MAE。第二类是 Top-N 推荐指标，包括 Precision@K、Recall@K 和 HitRate@K。本项目将测试集中评分不低于 4.0 的电影视为相关电影，考察模型推荐列表中命中相关电影的比例、覆盖程度以及用户层面的命中情况。

需要说明的是，评分预测误差和 Top-N 推荐命中率并不总是同步变化。一个模型可能更擅长预测具体评分值，因此 RMSE 和 MAE 较低；另一个模型可能更容易把用户喜欢的电影排到推荐列表前列，因此 Precision@K、Recall@K 或 HitRate@K 较高。因此，本项目同时报告两类指标，并通过 top_k 敏感性分析和多随机种子稳定性评估减少单次划分带来的偶然性。

5. 实验结果与分析

模型对比实验记录在 `notebooks/model_comparison.ipynb` 中。当前版本直接进行全量用户评估，不再把 20 用户快速演示作为最终对比结论。在随机种子 42、测试比例 0.2、推荐列表长度 Top-N=10、邻居数量 `top_k_neighbors=20` 的设置下，两个模型在同一份训练/测试划分上运行，结果如下。

表 2 全量模型评价指标对比

模型	用户数	评分数	RMSE	MAE	加权 RMSE	Precision@K	Recall@K	HitRate@K
NumPy 基线	475	14093	0.822211	0.661575	0.828074	0.060842	0.030942	0.336842
优化主模型	475	14093	0.798196	0.638230	0.802795	0.064842	0.028754	0.317895

从评分预测误差看，优化后的主模型 `sklearn_centered_item_cf` 的 RMSE 为 0.798196，MAE 为 0.638230，均低于 NumPy 基线模型的 RMSE 0.822211 和 MAE 0.661575。这说明用户均值中心化、相似度置信度约束和预测裁剪能够降低评分预测误差。

从 Top-N 推荐命中指标看，优化后的主模型 Precision@K 为 0.064842，高于 NumPy 基线模型的 0.060842；但 `baseline_numpy_item_cf` 的 Recall@K 和 HitRate@K 分别为 0.030942 和 0.336842，略高于主模型的 0.028754 和 0.317895。这表明排序层改善了推荐列表前部的平均精度，但相关电影覆盖率和用户层命中率仍有提升空间，两类指标反映的是不同能力。

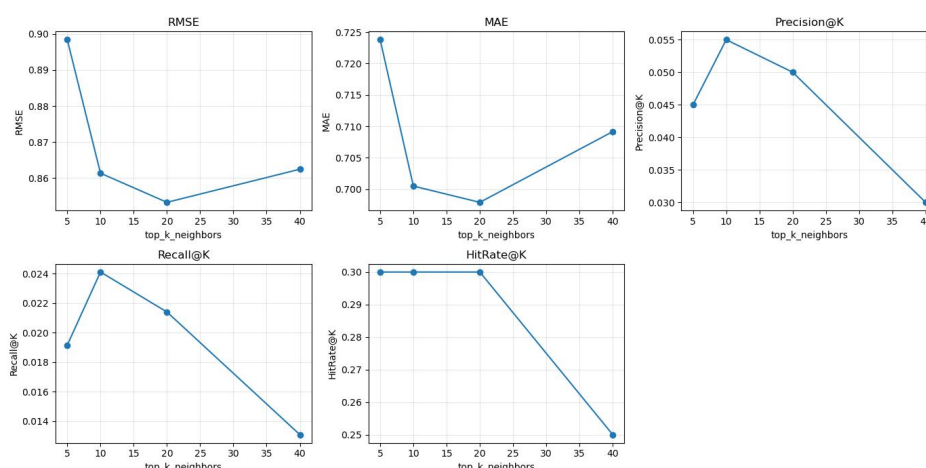


图 4 主模型快速演示评估指标

该图来自 `model_evaluation_visualization.ipynb`，仅用于说明评估流程，不作为最终实验结论。

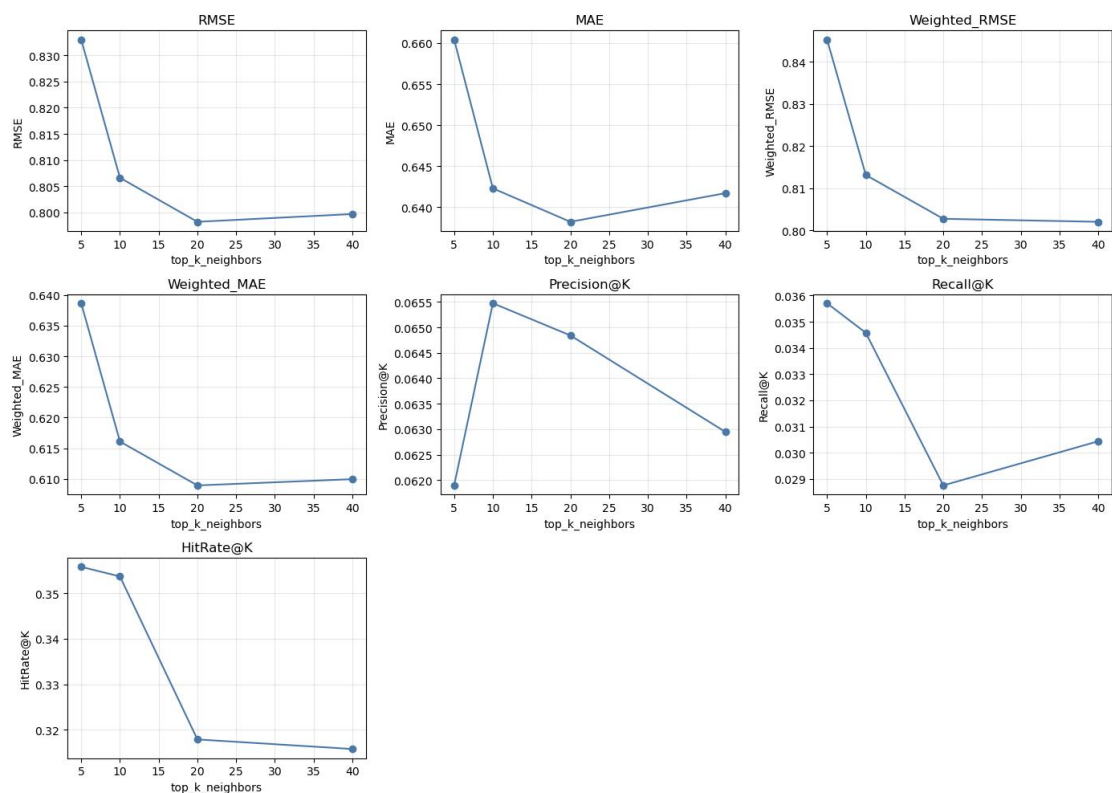


图 5 主模型全量评估指标

该图来自 `model_evaluation_visualization.ipynb`，展示优化后主模型在全量用户上的正式评估结果。

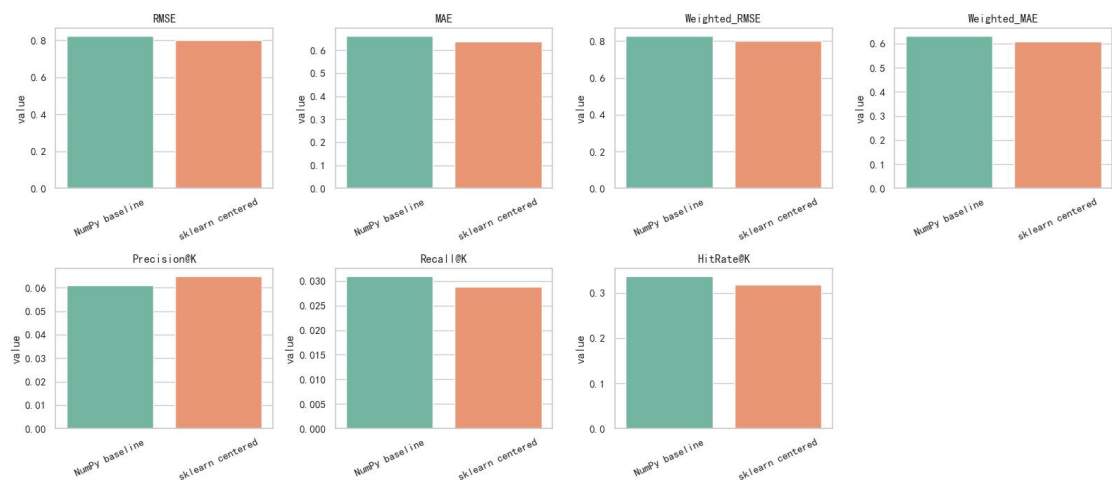


图 6 全量模型评价指标柱状图

该图来自 `model_comparison.ipynb`，展示 `baseline_numpy_item_cf` 与优化后主模型的全量指标对比。

为了进一步分析邻居数量对模型效果的影响，项目进行了全量 top_k 敏感性分析，分别测试 top_k=5、10、20、40。结果如下。

表 3 全量 top_k 敏感性分析结果

模型	top_k	RMSE	MAE	Precision@K	Recall@K	HitRate@K
NumPy 基线	5	0.843858	0.669305	0.050947	0.029356	0.303158
NumPy 基线	10	0.825110	0.660476	0.059368	0.031347	0.324211
NumPy 基线	20	0.822211	0.661575	0.060842	0.030942	0.336842
NumPy 基线	40	0.832254	0.672970	0.053053	0.024301	0.269474
优化主模型	5	0.833013	0.660409	0.061895	0.035708	0.355789
优化主模型	10	0.806611	0.642302	0.065474	0.034578	0.353684
优化主模型	20	0.798196	0.638230	0.064842	0.028754	0.317895
优化主模型	40	0.799682	0.641716	0.062947	0.030439	0.315790

从表 3 可以看出，邻居数量并非越大越好。baseline_numpy_item_cf 在 top_k=20 时 RMSE、Precision@K 和 HitRate@K 综合表现较好；优化后的主模型在 top_k=20 时 RMSE 和 MAE 最低，在 top_k=10 时 Precision@K 和 HitRate@K 较高，说明评分预测误差与 Top-N 排序效果之间存在权衡，参数选择需要结合最终应用目标。

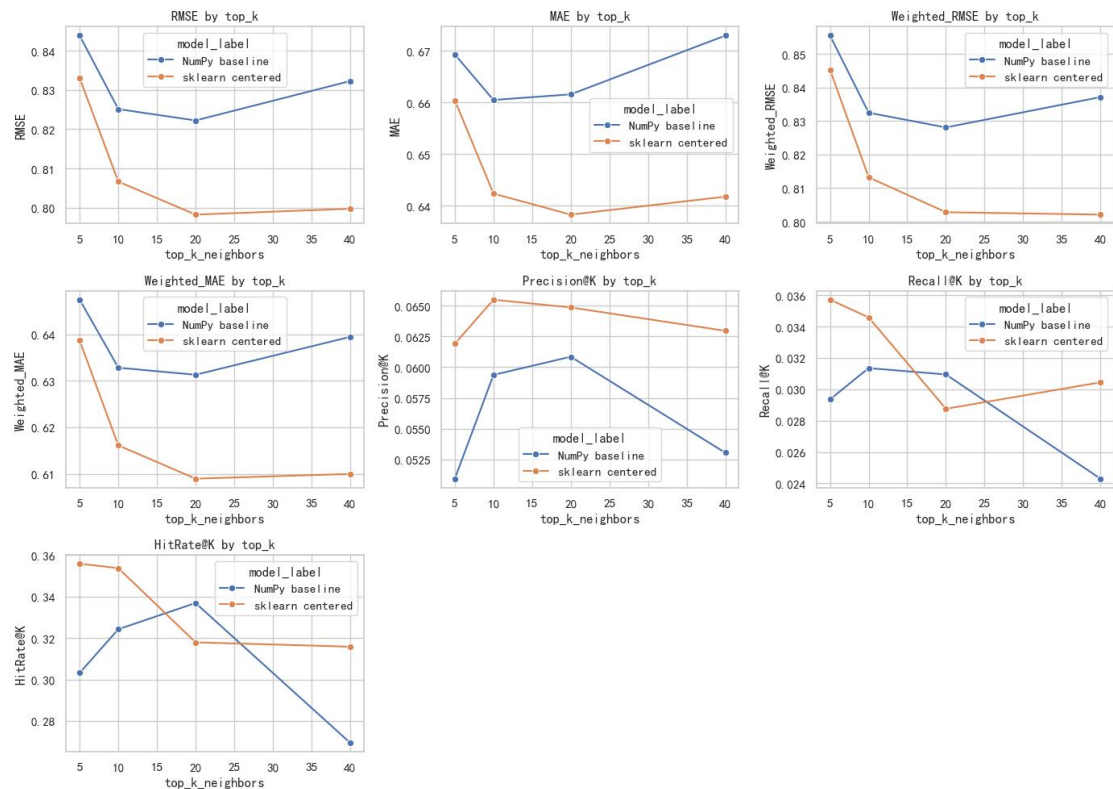


图 7 全量 top_k 敏感性分析折线图

该图展示不同邻居数量下两个模型多项指标的变化趋势，可用于说明参数选择对协同过滤效果的影响。

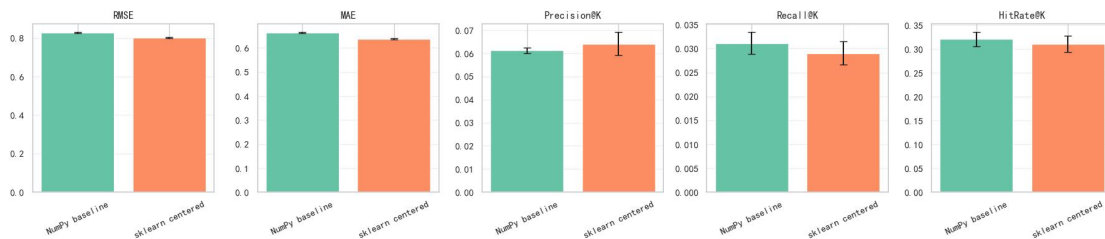


图 8 多随机种子稳定性评估

该图来自 `model_comparison.ipynb`，用于观察不同随机划分下主要指标的波动情况。

6. 案例分析

为了展示模型的实际推荐效果，项目在 `model_comparison.ipynb` 中选取样例用户，分别输出两个模型的 Top-10 推荐电影，并通过 `cleaned_movies.csv` 或原始 `movies.csv` 将 `movieId` 映射为电影标题。该案例分析的作用是说明推荐系统并非只输出抽象指标，而是能够为具体用户生成可解释的电影列表。

从推荐逻辑看，系统首先读取样例用户在训练集中的历史评分，找到该用户已经评价过的电影；然后对每一部未评分电影计算预测评分，并在主模型中结合热门度和相似度证据强度进行轻量重排序；最后按照排序分数从高到低输出 Top-10 推荐。对于 NumPy 基线模型，推荐结果更直接地受原始评分和热门电影相似度影响；对于优化后的主模型，推荐结果同时考虑用户评分尺度、相似度置信度和推荐排序证据。

表 4 指定用户 Top-10 推荐结果。样例用户为 `userId=1`，结果来自 `model_comparison.ipynb` 的推荐案例输出；推荐分数为模型对候选电影的预测评分。

模型	排名	movieId	电影标题	推荐分数
NumPy 基线	1	4878	Donnie Darko (2001)	4.9099
NumPy 基线	2	68358	Star Trek (2009)	4.9064
NumPy 基线	3	91529	Dark Knight Rises, The (2012)	4.8701
NumPy 基线	4	54272	Simpsons Movie, The (2007)	4.8594
NumPy 基线	5	778	Trainspotting (1996)	4.8590
NumPy 基线	6	4011	Snatch (2000)	4.8583
NumPy 基线	7	1221	Godfather: Part II, The (1974)	4.8544
NumPy 基线	8	7361	Eternal Sunshine of the Spotless Mind (2004)	4.8469
NumPy 基线	9	6711	Lost in Translation (2003)	4.8452
NumPy 基线	10	111659	Maleficent (2014)	4.8390
优化主模型	1	6874	Kill Bill: Vol. 1 (2003)	4.9597
优化主模型	2	68358	Star Trek (2009)	4.9518
优化主模型	3	8784	Garden State (2004)	4.9469
优化主模型	4	55765	American Gangster (2007)	4.9317
优化主模型	5	4973	Amelie (Fabuleux destin d'Amélie Poulain, Le) (2001)	4.9225

优化主模型	6	3252	Scent of a Woman (1992)	4.9180
优化主模型	7	1210	Star Wars: Episode VI - Return of the Jedi (1983)	4.9076
优化主模型	8	58998	Forgetting Sarah Marshall (2008)	4.9068
优化主模型	9	4963	Ocean's Eleven (2001)	4.9033
优化主模型	10	296	Pulp Fiction (1994)	4.9008

除 `notebook` 中的离线实验外，项目还提供了基于 `Streamlit` 的交互式展示页面 `demo.py`。该页面将推荐系统包装成一个简化电影网站原型，用户可以浏览热门电影、高分电影和类型电影，也可以按片名搜索电影详情，并在“我的推荐”页面选择已有 `userId` 或手动输入 `userId` 获取个性化推荐结果。

从功能定位看，`Streamlit` 页面主要承担工程展示与交互验证作用：一方面，它把 `MovieLens` 数据规模、电影列表、用户画像和推荐结果以可视化界面呈现出来；另一方面，它调用当前优化后的主模型进行个性化推荐，使评估 `notebook` 中的算法结果能够落到真实可操作的页面流程中。该部分说明系统不只是离线计算指标，也具备面向用户展示推荐结果的雏形。

关于数据规模，本项目使用的 `MovieLens ml-latest-small` 数据对于课程项目和协同过滤实验是足够的：原始数据包含 610 名用户、9724 部电影和 100836 条评分，过滤后仍保留 475 名活跃用户、1617 部电影和 70454 条评分，能够支撑相似度计算、全量评估和交互式推荐展示。但从工业级推荐系统角度看，数据仍不算充分满意，因为过滤后稀疏率仍达到 90.83%，新用户、新电影和长尾电影推荐仍然依赖回退策略或有限相似度证据。

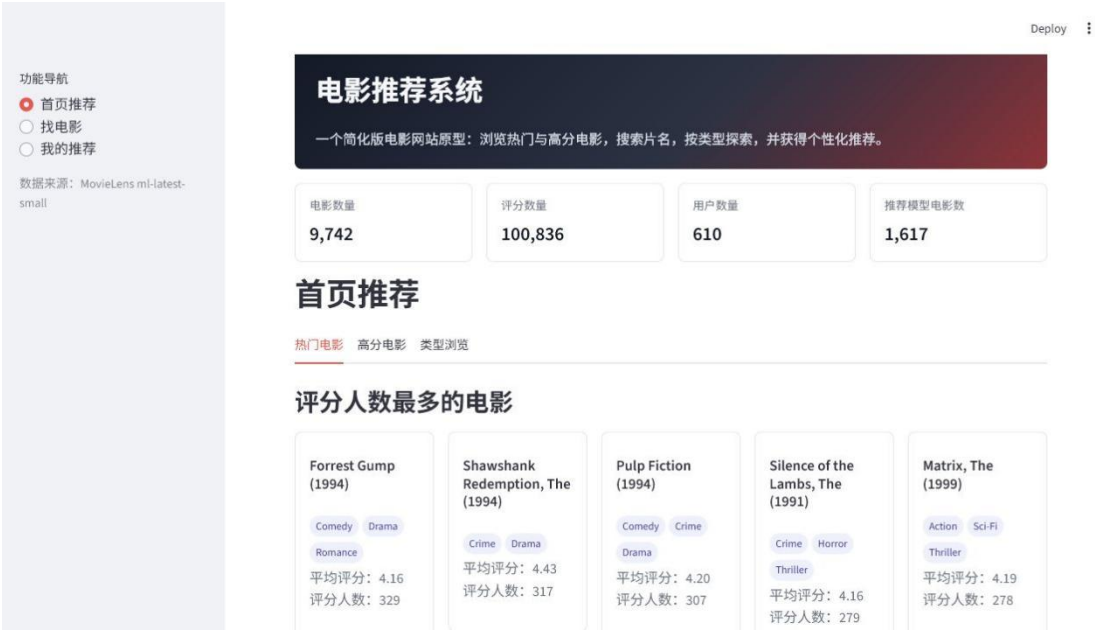


图 9 Streamlit 首页推荐界面

页面以电影网站形式展示数据规模、热门电影卡片和功能导航，用于说明系统具备可交互展示能力。



图 10 Streamlit 个性化推荐结果界面

用户选择 `userId` 后，系统展示历史评分信息与基于优化后主模型生成的 Top-N 推荐结果。

7. 不足与改进方向

本项目虽然完成了推荐系统的基本闭环，但仍存在一些不足。首先，评分矩阵在过滤后稀疏率仍然达到 90.83%，说明用户与电影之间的已知交互仍然有限。虽然当前主模型加入了相似度 `shrinkage` 和最低共评人数约束，但协同过滤仍然依赖历史评分，当用户评分较少或电影评分较少时，模型仍难以获得可靠相似度。其次，当前冷启动处理较简单，对于新用户或新电影主要依赖电影平均分或全局平均分回退。

再次，当前模型主要利用评分矩阵，没有充分利用 `movies.csv` 中的 `genres` 和 `tags.csv` 中的文本标签。未来可以将协同过滤与基于内容的推荐结合，利用电影类型、标签和文本特征补充评分矩阵的不足。在评估层面，后续还可以加入 `NDCG@K`、`MAP@K`、覆盖率、多样性和新颖性等排序指标，并扩大多随机种子评估范围。

在模型方面，后续可以尝试矩阵分解方法，例如 `SVD` 或基于 `Surprise` 库的推荐算法，也可以引入隐语义模型学习用户和电影的低维向量表示。此外，可以继续调参相似度 `shrinkage`、最低共评人数阈值、热门度权重和置信度权重，并考虑按用户活跃度或电影流行度分组评估模型表现。

8. 结论

本文围绕 `MovieLens` 数据集构建了一个基于协同过滤的电影推荐系统，完成了从数据清洗、探索性分析、稀疏矩阵过滤到模型训练与评价的完整实践流程。项目通过评分分布、时间趋势和矩阵热力图等可视化方法分析了数据特征，并采用活跃用户与热门电影筛选策略将矩阵稀疏率从 98.30% 降低到 90.83%，为后续建模提供了更合适的数据基础。

在模型部分，项目实现了 `NumPy` 手写基线模型和优化后的 `sklearn_centered_item_cf` 主模型，并在统一的训练/测试划分下进行全量对比实验。结果表明，优化后的主模型在 `RMSE` 和 `MAE` 上优于 `NumPy` 基线，同时 `Precision@K` 也略有提升；但基线模型在 `Recall@K` 和 `HitRate@K` 上仍略高，说明评分预测精度、推荐列表前部精度和相关电影覆盖率并不完全等价。因此，最终实验结论应同时参考误差指标和 Top-N 指标，而不能只依据单一指标判断模型优劣。

参考文献

- [1] F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 2015.
- [2] GroupLens Research. MovieLens Datasets. <https://grouplens.org/datasets/movielens/>
- [3] scikit-learn Developers. sklearn.metrics.pairwise.cosine_similarity Documentation. <https://scikit-learn.org/>
- [4] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez. Recommender systems survey. *Knowledge-Based Systems*, 2013.
- [5] Pratik Nabriya. Recommender-System-Collaborative-Filtering-MovieLens. GitHub repository.
- [6] Abhi Patel. MovieMatcher-Movie-Recommender-System. GitHub repository.

附录：项目文件与复现说明

本项目主要文件如下：

文件	作用
src/data_clean.py	原始 CSV 数据清洗，输出 cleaned 数据和日志
src/data_processing.py	构建用户-电影评分矩阵
src/model.py	NumPy 手写基线协同过滤模型
src/cf_recommender.py	优化后的主模型：中心化物品协同过滤、相似度置信度约束和 Top-N 重排序
src/evaluation.py	训练/测试划分、全量评估、Top-N 指标和模型评价指标计算
notebooks/analysis.ipynb	数据分析、稀疏矩阵处理和可视化
notebooks/model_evaluation_visualization.ipynb	主模型快速演示、全量评估、top_k 敏感性分析和评估结果可视化
notebooks/model_comparison.ipynb	两个模型全量对比、top_k 敏感性分析、多随机种子稳定性评估和推荐案例
data/cleaned_data/	清洗后数据和过滤后的评分矩阵

复现实验时，先安装 requirements.txt 中依赖，然后按顺序运行数据清洗、矩阵构建和 notebook。若只复现实验结果，可直接运行 notebooks/model_evaluation_visualization.ipynb 查看主模型快速演示与全量评估，运行 notebooks/model_comparison.ipynb 查看全量模型对比、top_k 敏感性分析、多随机种子稳定性评估和指定用户推荐案例。